Date:

Question no 1

4.1

1 Web server handling multiple clients
2 File downloading + UI responsiveness
3 Parallel matrix multiplication

4.2

(a) $N = 2$

$$\text{speedup} = \frac{1}{(0.4 + 0.6/2)} = 1.43$$

(b) $N = 4$

$$\text{speedup} = \frac{1}{(0.4 + 0.6/4)} = 1.82$$

4.3

Task parallelism (different tasks: handling requests)

4.4

- User threads: managed by user, faster, no kernel ~~sy~~ support
- Kernel threads: managed by OS, slower but true parallelism Better
- User → lightweight tasks
- Kernel → multiprocessor systems

4.5

- Save current thread state (registers, PC)
- Update PCB/TCB
- Load next thread state
- Switch execution

4.6

Thread:
- stack, registers, PC

Process:
- Address space, heap, stack, files

→ Threads share resources; processes don't

4.7

Yes, real-time thread should bind to LWP

Reason:

- guarantees CPU scheduling and timing

4.8

- Small tasks (thread overhead > benefit)
- Sequential dependency (eg recursive Fibonacci)

4.9

When one thread blocks (I/O), others continue - better CPU utilization

4.10

Shared:

b Heap

c Global variables

Not shared:

a Registers

d ~~b~~ Stack

4.12

No

Threads share ~~mem~~ memory → crash affects all tabs

Processes provide isolation → safer

4.11

Yes, because multiple CPUs execute threads in parallel

4.13

Yes

Concurrency = interleaving

Parallelism = simultaneous execution

4.14

i $P = 0.4$

a ~~i~~ $N = 8 \rightarrow \frac{1}{0.6 + 0.05} = 1.54$

Pg No.

b $N = 16 \rightarrow \frac{1}{(0.6 + 0.025)} = 1.6$

ii $P = 0.67$

a $N = 2 \rightarrow \frac{1}{(0.33 + 0.335)} = 1.50$

b $N = 4 \rightarrow \frac{1}{(0.33 + 0.1675)} = 2.01$

iii $P = 0.9$

a $N = 4 \rightarrow \frac{1}{(0.1 + 0.225)} = 3.08$

b $N = 8 \rightarrow \frac{1}{(0.1 + 0.1125)} = 4.71$

4.15

- Thumbnail generation → Task
- Matrix transpose → Data
- Network read/write → Task
- Fork-join sum → Data
- GCD system → Task

4.16

- I/O threads → 1 thread (only at start/end)
- CPU threads → 4 threads (equal to cores)

4.17

Code analysis

- Processes = 4
- Threads = 1 extra thread (only in one child)

4.18

- Linux → treats threads and processes same (flexible, lightweight)
- Windows → separate structures (clear distinction controlled)

4.19

Output

- Child: value = 5
- Parent: value = 0 (parent has separate memory after fork)

4.22

- Create 3 threads
  - Thread 1 → average
  - Thread 2 → minimum
  - Thread 3 → maximum
- Store results in global variables
- Parent prints results